



Shift-Left Requirements Verification: Integrating LLMs and Formal Methods for Automotive Systems

Zipong Lim^{1,2} , Bozhi Wu³ , Yon Shin Teo² , Shang-wei Lin⁴ ,
and Yi Li¹

¹ Nanyang Technological University, Singapore, Singapore
zipong001@e.ntu.edu.sg, yi_li@ntu.edu.sg

² AUMOVIO Singapore Pte. Ltd., Singapore, Singapore
{zi.pong.lim,yon.shin.teo}@aumovio.com

³ AUMOVIO-NTU Corporate Lab, Singapore, Singapore
bozhi001@e.ntu.edu.sg

⁴ Singapore Institute of Technology, Singapore, Singapore
shangwei.lin@singaporetech.edu.sg

Abstract. Requirement defects are a major source of late-stage failures in automotive systems, yet rigorous validation is rarely applied during early development. While formal methods offer strong guarantees, their adoption at the requirements level is limited by high formalization cost and expertise barriers. We present an industry-oriented, shift-left verification approach that integrates Large Language Models (LLMs) with formal methods to enable requirements-level validation. Requirements are classified and decomposed by LLMs, translated into CSP system models and assertions, and refined through a CEGAR-inspired loop using the FDR4 model checker. Validation is decomposed into requirement–assertion pairs supported by natural-language back-translations and confidence scores, preserving expert control without manual formal modeling. Domain knowledge-based validation further leverages historical defect data to identify implicit requirement gaps. We evaluate the approach on three real-world automotive case studies. Results show high automation for small-to-medium systems (80–100% synthesis success for up to ~ 60 requirements), effective expert validation guided by LLM confidence estimates, and 100% detection of known historical defects alongside 22 novel gaps. The workflow completes within 10–35 min per project at negligible cost (< 8 per project), with limited expert effort. Our results demonstrate that LLM+formal method hybridization can provide scalable, rigorous, and industrially viable requirements-level verification, supporting practical shift-left adoption in automotive systems.

Keywords: Requirements Engineering · Requirements Verification · Formal Methods · Large Language Models · Model Checking · CSP · Automotive System

1 Introduction

As a Tier-1 automotive supplier, **Aumovio** (formerly Continental Automotive) operates within a complex supply chain in which system requirements are primarily provided by Original Equipment Manufacturers (OEMs). Based on these documents, Aumovio develops safety-critical automotive systems that must meet stringent correctness and compliance requirements. However, at the requirements stage, no formal system model typically exists. Although requirement specifications are extensive and carefully written, they frequently contain ambiguities, inconsistencies, and implicit assumptions that remain undetected until later development phases. Without early-stage validation, such latent defects propagate downstream and often surface only during integration testing or system validation, where their correction incurs substantial cost and risk.

Automotive development traditionally follows the V-model, in which requirements engineering occurs at the earliest phase, while verification and validation activities are deferred to later stages. Decades of industrial experience show that defects discovered during integration or system testing are one to two orders of magnitude more expensive to fix than those detected during requirements engineering [3,24]. Importantly, many late-stage failures are not caused by implementation errors, but by deficiencies in the requirements themselves, including vague formulations, hidden inconsistencies, missing corner cases, and undocumented assumptions. These observations motivate a shift-left approach that brings rigorous verification techniques into the requirements phase.

Despite its importance, requirements verification faces a fundamental tension between scalability and assurance. Industrial specifications are large, informal, and evolving, making manual formalization prohibitively expensive, while safety-critical systems demand correctness guarantees that cannot be compromised.

On one hand, formal methods offer strong assurance by enabling mathematically precise reasoning about system behavior, and they have been successfully applied at the software and design levels in automotive development [6,8,14]. However, their adoption at the requirements stage remains challenging due to the *formalization barrier*: manual translation of informal text into formal specifications requires dual expertise in both domain knowledge and formal methods, which is a combination rarely available in industrial settings. Domain experts with deep system knowledge often lack formal methods training, while formal methods specialists often lack domain expertise. This skills gap makes formal verification approaches difficult to adopt at the requirements phase. On the other hand, recent works [1,2,12] leverage Large Language Models (LLMs) to automate requirements analysis, addressing the scalability of the problem. While LLMs excel at processing large volumes of text, they provide no inherent correctness guarantees: their outputs may be fluent and plausible yet semantically incorrect, making them unsuitable as the sole basis for verification in safety-critical contexts. These complementary strengths and weaknesses suggest that neither formal methods nor LLMs alone can adequately address requirements-level verification. Instead, an effective solution must combine the automation and scalability of LLMs with the soundness and rigor of formal methods.

Therefore, we propose a practical hybrid methodology that integrates LLMs with formal methods to enable scalable yet trustworthy requirements verification in industrial automotive settings. Our key idea is to treat LLMs as automation assistants, responsible for accelerating transformation tasks, while delegating all correctness-critical decisions to established formal techniques, specifically Communicating Sequential Processes (CSP) [13] and the FDR4 model checker [10].

The workflow supports *shift-left* verification where free-form natural-language requirements are automatically transformed into formal assertions, checked for correctness and consistency, and used to synthesize a verified system model before architectural or implementation decisions are finalized. Critically, the approach is human-in-the-loop: domain experts retain control over high-level validation while repetitive formal reasoning tasks are automated.

A central technical element is an iterative, counterexample-guided refinement loop inspired by CEGAR. Starting from an initial system model synthesized from validated assertions, all properties are checked in parallel using FDR4. When violations are detected, concrete counterexample traces are fed back to the LLM, which refines the model while preserving previously satisfied properties. To reduce latent requirement gaps, we integrate retrieval-augmented verification that leverages institutional knowledge such as historical defect reports, proactively identifying missing or underspecified requirements.

In summary, this paper makes the following contributions¹:

- An end-to-end industrial workflow synthesizing formal CSP system models from free-form natural-language requirements through human-LLM-FM collaboration, employing a CEGAR-inspired refinement methodology where LLMs iteratively repair system models based on FDR4 counterexamples.
- A decomposed validation approach transforming the cognitively demanding task of validating complex system models into manageable assertion-requirement pair reviews, combined with retrieval-augmented verification exploiting historical defect knowledge.
- Empirical evaluation on real-world automotive projects demonstrating substantial reductions in expert effort and effective early defect detection.
- Successful trial deployment with an industry partner OEM, where our tool identified a better fix for a previously known issue, demonstrating practical value beyond defect detection to include solution recommendation.

2 Motivation

Automotive systems are typically developed under the V-model, where requirements engineering is expected to provide a correct foundation for all downstream tasks. In practice, however, requirement-related defects remain a major source of late-stage failures. Defects introduced during requirements engineering often

¹ Due to space constraints, additional details and examples can be found in the online appendix: <https://sites.google.com/view/shift-left-requirement-suppl>.

propagate into implementation and are only discovered during integration testing or operation, where remediation costs are substantially higher. Despite informal reviews, requirements are rarely subjected to rigorous validation before being used for development, resulting in a persistent requirements validation gap.

Formal methods offer a principled means to bridge this gap by enabling mathematical validation of system behavior against requirements. However, their industrial adoption in automotive practice remains limited and restricted to later development stages, where verification is applied to artifacts such as models or code. Applying formal methods directly at the requirements level is therefore desirable for shift-left verification, but introduces several fundamental challenges.

Challenge 1: Formalization Barrier. Traditional formal methods rely on manually translating natural-language requirements into formal specifications, requiring expertise in both the application domain and formal modeling languages. In industrial automotive settings, such dual expertise is rarely available: domain experts typically lack formal methods training, while formal methods specialists often lack detailed system knowledge. This mismatch confines formal verification to late development stages and prevents its effective use during requirements engineering.

Challenge 2: Implicit Knowledge Problem. Requirements documents primarily capture *explicit* stakeholder intent and rarely document implicit assumptions, edge cases, or lessons learned from past failures. Consequently, even a formally verified specification may permit undesirable behaviors simply because critical domain constraints were never stated as requirements. In practice, organizations accumulate substantial domain knowledge in the form of historical defect reports, safety standards, and engineering experience, yet this knowledge remains largely tacit and disconnected from requirements-level validation.

Challenge 3: Formal Model Validity. These challenges converge on a fundamental question: how can the *validity* of a formal model be established? As illustrated in Fig. 1, formal specifications must reflect not only stated system requirements R but also relevant domain knowledge D . Traditional formal methods primarily address the $R \rightarrow S$ relationship, ensuring that specifications encode documented requirements, while often neglecting the $D \rightarrow S$ relationship. Many late-stage defects originate from this gap, where a specification satisfies all stated requirements but violates implicit domain constraints.

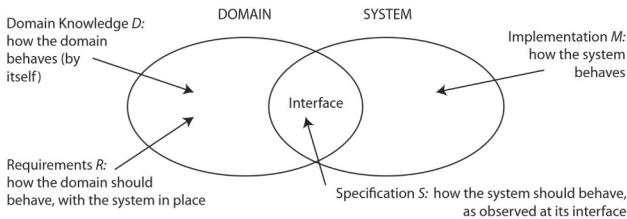


Fig. 1. Relationship between domain knowledge D , requirements R , specification S , and implementation M , adapted from [25].

Our Approach. To address these challenges, we propose an industry-oriented LLM+FM hybrid approach following the principle: LLMs for automation, formal methods for verification, humans for validation. For challenge 1, we decompose validation into assertion-requirement pair reviews with LLM-generated back-translations, shifting human expert effort from low-level formal reasoning to high-level semantic validation. For challenge 2, retrieval-augmented validation transforms tacit organizational knowledge into actionable verification properties. For challenge 3, we address both the $R \rightarrow S$ relationship (through decomposed assertion-requirement reviews) and the $D \rightarrow S$ relationship (through retrieval-augmented validation against historical defect knowledge).

3 Methodology

Our methodology consists of three phases that incrementally construct and validate a formal system specification, with human experts involved at key decision points, as illustrated in Fig. 2.

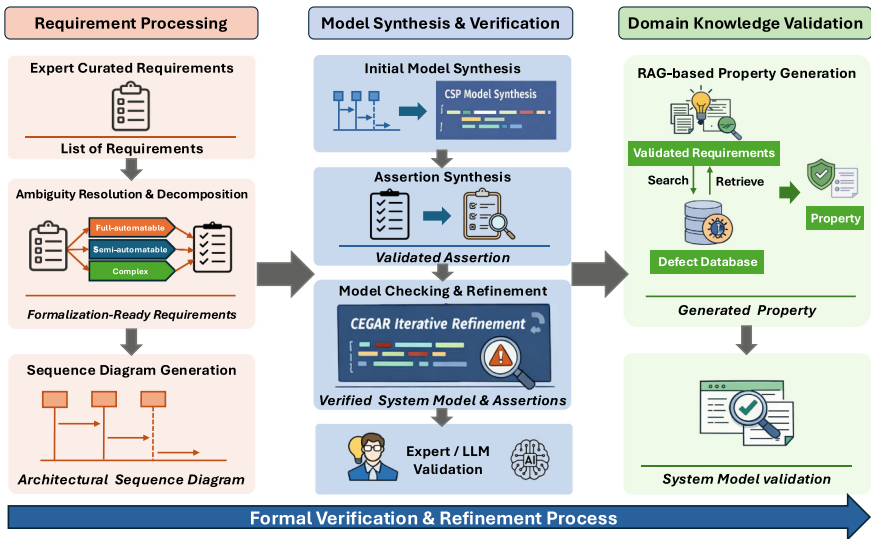


Fig. 2. Overview of the LLM+FM hybrid verification workflow.

3.1 Requirement Processing

OEM requirement documents typically contain heterogeneous content, including tables, flowcharts, diagrams, and free-form text. To extract concrete requirements from such multimodal artifacts, Aumovio employs an internally developed

AI-assisted toolchain based on Large Language Models (LLMs) and Vision–Language Models (VLMs), which converts complex documents into a list of textual requirements. Before formalization, however, these requirements must be further processed to ensure semantic clarity and modeling tractability.

Specifically, we employ an LLM (prompt provided in [Appendix L.1](#)) to classify each requirement according to its formalizability into three categories: *Fully Automatable (FA)* requirements that are clear and can be directly formalized; *Semi-Automatable (SA)* requirements that contain ambiguities requiring expert clarification; and *Complex (CX)* requirements that involve substantial domain complexity or multiple interacting concerns. The LLM output is subsequently reviewed and confirmed by domain experts, combining LLM automation efficiency with human validation for reliability. Ambiguities identified in SA requirements are resolved through expert clarification, while CX requirements are decomposed into simpler sub-requirements using a divide-and-conquer strategy. We now have a set of processed requirements that have been classified, disambiguated, and decomposed as needed, suitable for formal modeling.

After ambiguity resolution and decomposition, we construct an intermediate architectural abstraction in the form of a sequence diagram. At this stage, the LLM analyzes the processed requirements to identify system components, message flows, and temporal ordering constraints, and generates a textual sequence diagram (e.g., in PlantUML or Mermaid). This representation is validated by domain experts and serves as a structured bridge between natural-language requirements and subsequent formal CSP modeling. Examples of requirement processing are provided in [Appendix A](#) and [Appendix B](#).

3.2 Model and Assertion Synthesis

Initial Model Synthesis. After requirement processing, we obtain a set of processed requirements and a validated sequence diagram that serves as an architectural scaffold for model synthesis. By explicitly identifying components, channels, and causal message flows, the sequence diagram reduces architectural ambiguity. Given the diagram together with the processed requirements, we prompt LLMs ([Appendix L.2](#)) with CSP^M syntax documentation to synthesize an initial CSP system model M_0 .

Specifically, model construction follows a structured, diagram-guided workflow. First, project-specific datatypes and channels are declared to reflect the vocabulary introduced by the requirements. Next, CSP processes are synthesized to correspond to individual system components identified in the sequence diagram. Inter-component interactions are realized through channel synchronization, and the overall system behavior is defined as a parallel composition of these component processes with appropriate synchronization constraints. This ensures that the synthesized model preserves both the component boundaries and interaction semantics implied by the requirements and architectural context.

For requirements involving timing constraints (e.g., “within 200 ms”), they are abstracted into ordering constraints that are verifiable in untimed CSP. Rather than modeling explicit time, we preserve the essential causal dependencies

between events using trace refinement combined with alphabet hiding. For example, in CS1 (Sect. 4), to capture the requirement that `relay.ON` must occur after `ignition.ON`, we define: `AlphaSpec_G3 = {| ignition, relay |}`; `SPEC_G3 = ignition.ON -> relay.ON -> SPEC_G3`; `assert SPEC_G3 [T= SYSTEM diff(Events, AlphaSpec_G3)]`. Here, alphabet selection is performed by LLM based on the events referenced in each requirement, and alphabet hiding is used to restrict the refinement check to the relevant events, ensuring that the required ordering between `ignition.ON` and `relay.ON` is preserved in all system traces. During expert validation (Sect. 3.2), the back-translation explicitly describes which events are observed and which are hidden, enabling experts to verify that the chosen alphabet correctly captures the requirement’s scope. If experts identify that relevant events are incorrectly hidden, the assertion is flagged for revision.

Assertion Synthesis. Before model checking can be performed, the processed requirements must be translated into formal assertions that can be verified against the synthesized system model. To this end, we employ an LLM to translate all processed requirements into CSP assertions in a single batch operation (prompt provided in Appendix L.3). Batch synthesis ensures consistency across assertions and alignment with the datatypes, channels, and process structure introduced during system model synthesis. The generated assertions typically fall into several categories, including trace refinement for ordering constraints, failure refinement for availability properties, deadlock freedom, and determinism.

Following assertion generation, we perform explicit syntax validation of both the initial system model M_0 and the synthesized assertion set using the FDR4 syntax checker. Reported syntax errors, including undefined channels, datatype mismatches, and malformed expressions, are collected and provided to the LLM together with the relevant code fragments and diagnostic messages. The LLM is instructed to correct these errors, and this syntax refinement loop is repeated until all errors are resolved or a maximum of 20 attempts is reached, yielding a syntactically valid system model M_1 and assertion set $\mathcal{A}_1$.

CEGAR-Inspired Iterative Model and Assertion Refinement. After a syntactically valid system model M_1 and assertion set $\mathcal{A}_1$ are obtained, we refine both the model and assertions using a counterexample-guided refinement process inspired by CEGAR. To this end, all assertions $a_i \in \mathcal{A}_1$ are first checked against the current model in parallel, with a timeout of 60 s per assertion. Based on the verification results, the assertions are classified into three groups: $\mathcal{A}_{\text{pass}}$, which contains satisfied assertions; $\mathcal{A}_{\text{fail}}$, which includes violated assertions together with their counterexample traces; and $\mathcal{A}_{\text{timeout}}$, which consists of assertions that exceed the timeout due to state-space explosion.

Assertions in $\mathcal{A}_{\text{pass}}$ require no further action and are treated as invariants that must be preserved throughout subsequent refinement iterations. In contrast, for assertions in $\mathcal{A}_{\text{fail}}$, we apply a CEGAR-inspired global refinement strategy that processes all failures simultaneously in order to avoid local optimization traps, as detailed in Algorithm 1. In each refinement step, the `LLMREFINE` function constructs a prompt (Appendix L.4) containing the current model, all

failed assertions with their counterexamples and corresponding requirements, the set of passing assertions as constraints, and explicit refinement instructions. The LLM proposes updates to the model and the assertions (only when necessary), in a structured diff format. These updates are automatically parsed and applied to produce a refined model M' and assertion set $\mathcal{A}'$. After each update, all assertions are re-verified to detect potential regressions introduced by the refinement. This iterative refinement process continues until one of four termination conditions is met: (1) Success, when all assertions pass; (2) Oscillation, when the same pass-fail pattern recurs; (3) No progress, when the number of passing assertions does not increase for N consecutive iterations; or (4) Maximum iterations, when the limit is reached. Concrete refinement examples are provided in [Appendix C](#).

Assertions in $\mathcal{A}_{\text{timeout}}$ are handled separately through LLM-guided state-space reduction. For each such assertion, we guide the LLM ([Appendix L.4](#)) to simplify the model by removing processes not involved in the assertion alphabet, abstracting datatypes to smaller value sets, eliminating internal events not visible to the property, and applying process inlining. The resulting simplified model is then re-verified against the assertion, with up to 10 iterations with increasingly aggressive simplifications. Additional details of the procedure are provided in [Appendix D](#) and [L.5](#).

Expert Validation and Golden Standard Establishment. Once the refinement loop yields a system model satisfying all verifiable assertions, we validate assertion correctness through expert review. While refinement guarantees that the model satisfies the assertions ($M \models \mathcal{A}$), it does not ensure that the assertions faithfully capture the intent of the original requirements ($\mathcal{A} \equiv \mathcal{R}$). This validation addresses this gap through human-LLM collaboration.

For each verified assertion $a_i \in \mathcal{A}_{\text{verified}}$, experts review a structured package comprising the original requirement, the corresponding CSP assertion, an LLM-generated natural-language back-translation explaining what the assertion verifies ([Appendix L.6](#)), an LLM confidence score (0-1) with rationale ([Appendix L.7](#)), and the verification evidence. For example, from CS1 requirement G-2: “System shall provide four grille shutter opening degrees” was formalized as `SPEC_G2 = (actuator.FULL_OPEN -> SPEC_G2) [] (actuator.LEVEL_1 -> SPEC_G2) [] ...` with back-translation “This verifies all four degrees can be commanded” and confidence 0.85. Additional examples are in [Appendix E](#).

Experts assess each assertion-requirement pair for semantic alignment, completeness, abstraction adequacy, and case coverage. Assertions may be *Approved*, *Modified* (triggering re-verification), or *Rejected*. After review, all approved assertions are consolidated into the golden standard specification $\mathcal{A}_{\text{golden}}$, together with a traceability matrix, coverage report, and documented limitations.

By decomposing validation into requirement-assertion pairs rather than validating the system model monolithically, this process enables clear semantic traceability, supports incremental expert review, and provides a robust and scalable validation suitable for industrial practice.

Algorithm 1 Counterexample-Guided System Model and Assertion Refinement

```

1: Input: Model  $M$ , assertions  $\mathcal{A}$ 
2: Output: Refined  $M^*$  and  $\mathcal{A}^*$ , or stalemate report
3:  $iteration \leftarrow 0$ ,  $no\_progress \leftarrow 0$ ,  $history \leftarrow \emptyset$ 
4: while  $iteration < MAX\_ITER$  do
5:    $(\mathcal{A}_{pass}, \mathcal{A}_{fail}, \mathcal{A}_{timeout}) \leftarrow PARALLEL\_CHECK(M, \mathcal{A})$ 
6:   if  $\mathcal{A}_{fail} = \emptyset \wedge \mathcal{A}_{timeout} = \emptyset$  then
7:     return  $(M, \mathcal{A})$ 
8:   end if
9:    $pattern \leftarrow COMPUTE\_PATTERN(\mathcal{A}_{pass}, \mathcal{A}_{fail})$ 
10:  if  $pattern \in history$  then
11:    return  $STALEMATE("Oscillation", M, \mathcal{A}, \mathcal{A}_{fail})$ 
12:  end if
13:   $history \leftarrow history \cup \{pattern\}$ 
14:   $(M', \mathcal{A}') \leftarrow LLMREFINE(M, \mathcal{A}, \mathcal{A}_{fail}, \mathcal{A}_{pass})$ 
15:   $(\mathcal{A}'_{pass}, \mathcal{A}'_{fail}, \_) \leftarrow PARALLEL\_CHECK(M', \mathcal{A}')$ 
16:  if  $|\mathcal{A}'_{pass}| \leq |\mathcal{A}_{pass}|$  then
17:     $no\_progress \leftarrow no\_progress + 1$ 
18:  else
19:     $no\_progress \leftarrow 0$ 
20:  end if
21:  if  $no\_progress \geq THRESHOLD$  then
22:    return  $STALEMATE("No progress", M', \mathcal{A}', \mathcal{A}'_{fail})$ 
23:  end if
24:   $M \leftarrow M', \mathcal{A} \leftarrow \mathcal{A}', iteration \leftarrow iteration + 1$ 
25: end while
26: return  $STALEMATE("Max iterations", M, \mathcal{A}, \mathcal{A}_{fail})$ 

```

Final System Model Refinement. With $\mathcal{A}_{golden}$ established, we perform final model refinement to ensure it satisfies this authoritative set. We re-run parallel verification of M^* against all $\mathcal{A}_{golden}$. The final refinement follows Algorithm 1 but with: **Golden standard priority**, under which all assertions are treated as inviolable; **No assertion modifications**, meaning that only the model may be refined; and **Stricter stalemate criteria**. When stalemate occurs, diagnostic reports are escalated to experts. Upon successful completion, this phase produces final verified model M_{final} with formal guarantee $\forall a_i \in \mathcal{A}_{golden}, M_{final} \models a_i$.

Two-Phase Refinement Design. A key design decision is the separation between *iterative refinement* and *golden standard establishment*. During iterative refinement, both the model and assertions may be modified to resolve verification failures. This flexibility is intentional as early-stage assertions may contain formalization errors or overly strict interpretations that do not reflect requirement intent. However, this flexibility introduces risks of assertion weakening, where the loop converges by relaxing properties rather than correcting the model. To mitigate this, we impose a strict two-phase structure: (1) during iterative refinement, modifications are permitted but all changes are logged and presented to experts; (2) once the golden standard is established through expert

validation, assertions become immutable and only model refinement is permitted. This separation ensures that any assertion modifications undergo explicit human review before being accepted as authoritative, preventing silent semantic drift.

3.3 Domain Knowledge-Based Validation

This phase addresses a critical limitation: even when a system model satisfies all explicitly stated requirements, it may still exhibit undesirable behaviors or fail to account for implicitly assumed scenarios. To mitigate this risk, we perform domain knowledge-based validation that leverages institutional knowledge, particularly historical defect reports, to identify requirement gaps.

RAG-Based Approach. We adopt a retrieval-augmented generation (RAG) workflow. Given the current requirements and the verified system model, relevant historical defect databases are retrieved based on semantic similarity. An LLM then generates candidate properties intended to prevent the recurrence of similar defects, formalizes them into CSP assertions, and checks them against the system model. Properties that fail verification are treated as potential requirement gaps.

Defect Database. The defect base is constructed from historical defect reports, lessons learned documents, and test failure reports. Each entry is embedded using pre-trained language models and stored in a vector database. To retrieve relevant defects, we construct a query from current requirements and retrieve top- K most similar defect reports ($K = 10$) using cosine similarity with relevance filtering (threshold > 0.7). Defect database structure is available in [Appendix F](#).

Property Generation and Validation. Retrieved defect reports are fed to an LLM (Prompt in [Appendix L.8](#)) with current requirements and golden standard assertions to generate candidate properties $\mathcal{P}_{\text{candidate}}$. Each candidate property includes a natural-language description, an explanation of which historical defect(s) motivated it, an assessment of whether it is already covered by existing requirements, and a severity rating. Domain experts review these properties and produce a validated set $\mathcal{P}_{\text{validated}}$.

Verification and Gap Analysis. Each property $p_i \in \mathcal{P}_{\text{validated}}$ is checked against M_{final} using FDR4. For each gap, we generate a report containing: executive summary, failed property description, historical context, counterexample analysis with FDR4 trace and LLM explanation (Prompt available in [Appendix L.9](#)), root cause assessment, and recommended actions. Example gap reports are provided in [Appendix G](#).

Stakeholder Decision-Making. Based on these reports, stakeholders decide whether to *accept and fix* the gap, *accept with alternative mitigation*, *reject* it as not applicable, or *defer* the decision. When gaps are accepted and addressed, new requirements are formalized and added to the golden standard specification, the system model is refined, and domain knowledge-based validation is re-executed. The process terminates when all high-priority gaps are resolved, no new gaps emerge, or practical constraints require freezing the specification.

Overall, this phase does not replace human validation but restructures it into a focused and efficient workflow. By grounding gap detection in historical evidence and validating candidate properties through formal verification, the approach systematically exposes implicit requirement gaps and complements requirements-level verification in industrial settings.

4 Evaluation

4.1 Experimental Setup

We evaluate our methodology through case studies on three real-world automotive projects, assessing its feasibility, correctness, defect discovery capability, and industrial practicality. Specifically, we address the following four research questions: **RQ1**: Can LLMs automatically generate correct CSP system models and assertions from natural language requirements within practical iteration limits? **RQ2**: Do LLM-generated assertions accurately capture requirement intent, and can LLM confidence scores predict formalization quality? **RQ3**: Can domain knowledge-based verification identify real requirement gaps by leveraging historical failure modes? **RQ4**: Is the LLM+FM hybrid approach practical for industrial use in terms of time, cost, and expert effort?

Dataset. Table 1 summarizes three real-world automotive projects with increasing system complexity and diverse functional scope. The cases cover representative subsystem types in V-model development, including local control logic, state-machine-based configuration, and distributed protocol interactions, providing a realistic and varied evaluation setting.

Configuration. All experiments use Claude 4.5 Sonnet as the LLM and FDR4 for model checking, with iteration limits of 20 for both syntax and model-checking refinement loops. Each case study is run five times for consistency. The iteration limit of 20 was determined empirically: beyond this threshold, failed runs consistently exhibited oscillation patterns where the same assertion pass/fail configurations recurred, indicating fundamental conflicts rather than insufficient refinement attempts. Increasing the limit further did not improve success rates for complex systems.

Table 1. Overview of Real-world Automotive Cases. (Details in [Appendix H](#))

Case	#Req	Complexity	Domain	Key Characteristics
CS1	23	Low	Active Grille Shutter Control	Actuator control, sensor integration, power state transitions, diagnostic fault code generation
CS2	56	Medium	Sunroof Auto-Identification	Automatic hardware detection, state-machine-based configuration, CAN communication, non-volatile memory persistence
CS3	83	High	Remote Services Gateway	Multi-ECU distributed communication, remote control protocol compliance, multi-step request-response flows with security handshaking

4.2 RQ1: Synthesis Feasibility

Table 2. Success Rates and Average Iteration Counts (MC = Model Checking).

Case Study	Success Rate	Syntax Iter.	MC Iter.	Total Iter.
CS1 (23 req.)	5/5 (100%)	1.0	5.8	6.8
CS2 (56 req.)	4/5 (80%)	1.6	5.4	7.0
CS3 (83 req.)	2/5 (40%)	4.6	13.4	18.0

Table 2 summarizes the synthesis success rates and average iteration counts across three automotive case studies. CS1 (23 requirements) achieved a 100% success rate in only 6.8 total iterations on average, demonstrating high reliability for small-scale systems. CS2 (56 requirements) reached an 80% success rate; the single failed run exhibited oscillation patterns caused by conflicting assertions rather than synthesis limitations. In contrast, CS3 (83 requirements) showed a reduced success rate of 40% with an average of 18.0 total iterations, reflecting the higher complexity of its distributed multi-ECU architecture. Failed runs in CS3 consistently entered stalemates where violated assertions no longer decreased despite further refinement. More iterations further highlight the increasing difficulty of large-scale synthesis.

Answer to RQ1: LLMs can reliably synthesize correct CSP system models for small-to-medium automotive projects (up to ~60 requirements), achieving success rates between 80% and 100%. For larger and structurally more complex projects (80+ requirements), synthesis success drops to around 40%. This reduced success rate reflects fundamental complexity rather than insufficient iterations: failed runs consistently exhibited oscillation patterns where conflicting assertions could not be simultaneously satisfied. For industrial adoption at larger scales, several mitigation strategies are viable: improved LLM capabilities, enhanced prompting strategies, hierarchical requirement decomposition, and targeted expert intervention for conflicting assertion resolution. The iterative refinement loop is essential, with most successful runs requiring 5-10 total iterations.

4.3 RQ2: Assertion Formalization Accuracy

Table 3 and Table 4 summarize the correctness of LLM-generated assertions and their relationship with confidence scores. Expert validation on CS1 shows that the majority of assertions correctly capture requirement intent: 14 out of 23 assertions (61%) required no changes, accurately modeling event ordering, state transitions, and causal relationships. An additional 22% required only minor adjustments, typically clarifying abstraction choices rather than correcting semantic

Table 3. Distribution of Confidence Scores for Generated Assertions.

Case Study	High (0.80–1.00)	Medium (0.60–0.79)	Low (<0.60)	Avg. Score
CS1 (23)	8 (35%)	11 (48%)	4 (17%)	0.75
CS2 (56)	24 (43%)	30 (54%)	2 (4%)	0.77
CS3 (83)	31 (37%)	50 (60%)	2 (2%)	0.77

errors. Only 17% required major revisions, indicating that substantially incorrect assertions are a minority.

Confidence scores provide supporting evidence of correctness. Assertions accepted without modification exhibit the highest average confidence (0.81), whereas those requiring major revisions have significantly lower confidence (0.59). For example, assertion G-2 was correctly formalized, received a high confidence score (0.85), and was unanimously approved by experts, while assertion G-16 received lower confidence (0.72) alongside an explicit LLM disclaimer that aligned with expert feedback. Detailed expert review data are provided in [Appendix J](#).

Answer to RQ2: Most LLM-generated assertions correctly capture requirement intent, with 61% requiring no expert changes and only a small fraction requiring major revisions. LLM confidence scores correlate strongly with correctness and effectively flag assertions needing closer expert review.

Table 4. Expert Validation Outcomes for CS1 with LLM Confidence Scores.

Category	Count	%	R1	R2	R3	Avg. Conf.
No change required	14	61%	16	18	18	0.81
Minor changes needed	5	22%	3	1	2	0.74
Major revisions required	4	17%	4	4	3	0.59

4.4 RQ3: Defect Detection Capability

Table 5 summarizes the results of domain knowledge-based validation across the three case studies. In total, 29 candidate properties were generated from retrieved historical defect reports. Of these, 23 properties failed FDR4 verification, indicating that the current system model permits behaviors that could lead to defects similar to historical failures. The remaining 6 properties passed verification, meaning the system already satisfies these constraints. The 79% actionable finding rate (23/29) represents properties where formal verification exposed concrete gaps; these were subsequently reviewed by domain experts who

Table 5. Results of Domain Knowledge-based Validation.

Case Study	Retrieved	Failed	Passed	Known Issues	Novel Issues
CS1	11	7	4	0/0	7
CS2	8	8	0	0/0	8
CS3	10	8	2	1/1 (100%)	7
Total	29	23	6	1/1 (100%)	22

confirmed their validity as genuine requirement deficiencies requiring attention. More importantly, the approach detected 1 out of 1 known historical defect (100%), while uncovering 22 previously undocumented gaps, demonstrating its effectiveness beyond reproducing known issues. Cross-case analysis reveals recurring requirement gaps, dominated by initialization omissions (65%) where safe or well-defined post-reset states are unspecified. Temporal ordering gaps (26%) arise from unconstrained execution order, while obligation-versus-capability gaps (22%) reflect missing mandatory conditions. A smaller fraction involves security and protocol gaps (9%), such as omitted handshakes or validation steps.

Known Defect Detection (CS3). In CS3, a historical integration defect occurred when control requests were issued before completion of a security handshake. While the original OEM fix relied on a fixed delay (500ms), our domain knowledge-based validation retrieved the defect and generated a property enforcing handshake completion before control operations. Verification against M_{final} revealed a violation, confirming that the system could bypass the handshake. Both Aumovio and OEM stakeholders agreed that the handshake-based property provides a superior, timing-independent solution. Details are in [Appendix K](#).

Novel Gap Examples. Among the 22 novel gaps: *CS1 Initialization Gap*: “After power-on, the system SHALL initialize to a known safe state (grille fully open) before accepting control commands” failed, revealing that the model allowed control commands immediately after power-on without guaranteed initialization. *CS2 State Persistence Gap*: “After diagnostic clear of sunroof presence, the system SHALL re-execute detection logic on next ignition cycle” failed, showing the model allowed the system to remain in an indeterminate state.

Answer to RQ3: Domain knowledge-based validation detected 100% of known historical issues and identified 22 novel requirement gaps, achieving a 79% actionable finding rate. The approach is particularly effective at surfacing initialization sequencing and temporal ordering gaps. Beyond defect detection, it enables formally justified improvements to system behavior, demonstrating value that extends beyond reproducing known failures.

Table 6. Objective Metrics for Industrial Practicality Assessment. Expected costs account for synthesis success rates from Table 2.

Case Study	Requirements	Pages	Time	API Cost	Exp. Cost
CS1	23	3	10 min	\$1.80 (\$0.078/req)	\$1.80
CS2	56	11	35 min	\$7.80 (\$0.139/req)	\$9.75
CS3	83	9	20 min	\$3.30 (\$0.040/req)	\$8.25
Avg.	54	7.7	22 min	\$4.30	\$6.60

4.5 RQ4: Industrial Practicality and Efficiency

Table 6 reports objective metrics assessing the industrial practicality of our method. Across the case studies, the automated pipeline completed within 10–35 min of wall-clock time, even for the largest project with 83 requirements. Notably, processing time scales sub-linearly with requirement document pages, indicating that the approach remains practical as system complexity increases. API costs are consistently low, with all cases remaining well below \$8 per successful run. The “Expected Cost” column accounts for synthesis failure probability (success rates from Table 2), computed as API Cost divided by success rate. Even with potential re-runs, expected costs remain below \$10 per project. In practice, failed synthesis attempts provide diagnostic information (oscillation patterns, conflicting assertions) that can guide targeted expert intervention, potentially reducing subsequent attempt costs.

Comparison with Traditional Workflows. Our approach provides: **Formal guarantees** (mathematical proof that $M_{\text{final}} \models \mathcal{A}_{\text{golden}}$), **Exhaustive checking** (FDR4 explores complete state space within computational limits), **Concrete counterexamples** (precise diagnostic traces guide fixes), and **Traceability** (full requirement-to-assertion-to-model mapping for regulatory compliance). These benefits come at minimal additional cost: < 1 hour expert validation time and < 8 API cost per project.

Answer to RQ4: The LLM+FM hybrid approach demonstrates strong industrial practicality: (1) automated processing completes within 10–35 minutes for projects with 23–83 requirements, (2) API costs remain low (< \$8 per project, < \$0.15 per requirement), (3) and processing time scales sub-linearly with document size. The approach shifts expert effort toward semantic validation rather than formal modeling, enabling practical adoption in industrial settings where domain experts typically lack formal methods training.

5 Related Work

Formal methods have a long history in safety-critical domains, with model checking [4, 5, 14, 20, 22], theorem proving [6, 8, 11, 18], and static analysis [7, 15, 17,

21] increasingly adopted at design and implementation levels in automotive, aerospace, and railways [10, 13]. Standards such as ISO 26262 drive formal technique adoption, yet applying these methods at the requirements level remains challenging due to the manual effort for translating requirements into formal specifications.

To reduce this burden, structured specification techniques have been proposed. Controlled natural languages such as Attempto Controlled English restrict grammar and vocabulary [9] to enable deterministic parsing into logic. Requirement boilerplates and patterns, including EARS-style requirements syntax [19] and temporal property specification patterns, provide templates for expressing common behavioral constraints [3, 24]. While these techniques improve consistency and analyzability, they impose strict syntactic discipline and require significant process changes and training.

Recent work has explored using LLMs for requirements engineering tasks, including requirements classification [2, 12] and ambiguity detection [1]. However, these approaches typically lack formal verification guarantees, relying on LLM outputs without rigorous validation. Our work differs fundamentally by treating LLMs as automation assistants rather than verification authorities, with all outputs validated through formal model checking or expert review.

Retrieval-augmented generation (RAG) has been applied in various software engineering contexts [16, 23], but its use for proactive requirement gap detection through historical defect analysis is, to our knowledge, novel. By combining RAG with formal verification, we enable systematic validation against institutional knowledge that would otherwise remain tacit.

6 Discussion and Limitations

Dependence on LLM Quality. The effectiveness of our approach depends on the capabilities of the underlying LLM. LLMs may misinterpret domain-specific terminology or struggle in complex refinement scenarios. To mitigate this risk, LLMs are never treated as trusted verification authorities: all outputs are validated through formal checking or expert review, ensuring that errors are exposed rather than remaining latent. As LLMs improve, we expect corresponding gains in workflow performance without changes to the formal backbone. We informally evaluated several LLMs and found that Claude 4.5 Sonnet provided the best results for CSP synthesis and refinement; however, we did not conduct systematic comparisons across models. Results may vary with different LLMs, and we acknowledge this as a reproducibility consideration.

Timing Abstraction Limitations. Our approach abstracts timing constraints (e.g., “within 200ms”) into ordering constraints verifiable in untimed CSP. This abstraction is appropriate for requirements where the essential property is causal ordering (e.g., “A must occur before B”), but may be insufficient for requirements where bounded response time is fundamental. Examples of requirements that may be misrepresented include: watchdog timeouts where the bound itself is safety-critical, jitter constraints requiring bounded variance, and rate-limiting

requirements specifying maximum event frequency. In our case studies, timing requirements were predominantly ordering-based, and expert validation did not identify cases where the untimed abstraction materially changed requirement semantics. However, for systems with strict real-time constraints, complementary verification using timed formal methods (e.g., Timed CSP, timed automata) may be necessary. We consider integration with timed verification as future work.

Quality and Coverage of Domain Knowledge. Retrieval-augmented verification is influenced by the quality and coverage of the domain knowledge base. Historical defect records may be incomplete or biased toward certain issue classes, limiting recall. In our evaluation, missed issues were primarily due to insufficient documentation or hardware-dependent failures. Nevertheless, even partial knowledge proved valuable, identifying both known and previously undocumented high-priority gaps, and is expected to improve as organizations accumulate experience.

Human-in-the-Loop Requirement. Although the workflow significantly reduces expert effort, human involvement remains necessary for validating assertions, resolving conflicts, and handling verification stalemates. The approach is designed to support, rather than replace, expert judgment by automating low-level reasoning and providing precise diagnostics.

Scalability and Abstraction. While the method performs well for small-to-medium projects, larger systems may require hierarchical decomposition and compositional verification. The reduced success rate for CS3 (40%) indicates that fully automated synthesis has limitations at scale; however, the diagnostic information from failed attempts (oscillation patterns, conflicting assertions) provides actionable guidance for expert intervention.

Communicating Results to Non-Formal Stakeholders. The current workflow produces formal CSP models and assertions as primary outputs. For stakeholders who rely on natural-language or graphical specifications, we provide several mechanisms: (1) natural-language back-translations accompany each assertion, explaining its meaning in domain terms; (2) FDR4 can generate state transition diagrams from verified models, providing visual representations of system behavior (see [Appendix I](#)); and (3) the traceability matrix links formal assertions back to original requirements.

Reproducibility and Artifact Availability. Our evaluation uses proprietary OEM requirements, an internal defect database, and commercial LLM APIs, which limits direct reproducibility. To support replication and adaptation, we provide: complete prompts for all LLM interactions ([Appendix L](#)), detailed examples of refinement interactions and gap reports ([Appendices C,G](#)), the defect database schema with anonymized examples ([Appendix F](#)), and expert review data with inter-reviewer analysis ([Appendix J](#)). The methodology is designed to be LLM-agnostic, though performance may vary across models.

7 Conclusion

This paper presented an industry-oriented, shift-left approach to requirements-level verification for automotive systems by integrating large language models with formal methods. The proposed workflow automatically translates natural-language requirements into CSP system models and assertions, refines them through a CEGAR-inspired loop using FDR4 counterexamples, and validates correctness via expert review supported by natural-language back-translations.

Evaluation on real-world automotive case studies shows that the approach is practical and effective for small-to-medium projects, achieving high synthesis success with limited expert effort. Domain knowledge-based validation further extends requirements verification by uncovering implicit gaps using historical defect knowledge. Overall, this work demonstrates that LLM+FM hybridization can enable scalable and trustworthy requirements-level verification without major process changes, providing a concrete step toward practical shift-left adoption in industrial settings.

Acknowledgements. This research/project is supported by A*STAR under the RIE2025 Industry Alignment Fund – Industry Collaboration Projects (IAF-ICP) Funding Initiative (Award: I2501E0045), as well as cash and in-kind contributions from the industry partner(s), and the Singapore Ministry of Education Academic Research Fund Tier 1 (RG12/23).

References

1. Bashir, S., et al.: Requirements ambiguity detection and explanation with LLMs: an industrial study. In: International Conference on Software Maintenance and Evolution (2025)
2. Binkhonain, M., Alfayez, R.: Are prompts all you need? Evaluating prompt-based large language models (LLM)s for software requirements classification. In: Requirements Engineering, pp. 1–21 (2025)
3. Boehm, B., Basili, V.R.: Software defect reduction top 10 list. *Found. Empir. Softw. Eng. Legacy Victor R. Basili* **426**(37), 426–431 (2005)
4. Clarke, E.M.: Model checking. In: Ramesh, S., Sivakumar, G. (eds.) *FSTTCS 1997*. LNCS, vol. 1346, pp. 54–56. Springer, Heidelberg (1997). <https://doi.org/10.1007/BFb0058022>
5. Clarke, E.M., Emerson, E.A., Sistla, A.P.: Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Trans. Program. Lang. Syst. (TOPLAS)* **8**(2), 244–263 (1986)
6. Cook, B., Kroening, D., Sharygina, N.: Accurate theorem proving for program verification. In: International Symposium On Leveraging Applications of Formal Methods, Verification and Validation, pp. 96–114. Springer (2004)
7. Cousot, P., Cousot, R.: Systematic design of program analysis frameworks. In: Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, pp. 269–282 (1979)
8. Darvas, A., Hähnle, R., Sands, D.: A theorem proving approach to analysis of secure information flow. In: International Conference on Security in Pervasive Computing, pp. 193–209. Springer (2005)

9. Fuchs, N.E., Schwitter, R.: Attempto controlled English (ace). arXiv preprint [cmp-lg/9603003](https://arxiv.org/abs/1603.0003) (1996)
10. Gibson-Robinson, T., Armstrong, P., Boulgakov, A., Roscoe, A.W.: FDR3 — a modern refinement checker for CSP. In: Ábrahám, E., Havelund, K., (eds.), *Tools and Algorithms for the Construction and Analysis of Systems*, vol. 8413, LNCS, pp. 187–201 (2014)
11. Green, C.: Application of theorem proving to problem solving. In: *Readings in Artificial Intelligence*, pp. 202–222. Elsevier (1981)
12. Hassani, S., Sabetzadeh, M., Amyot, D.: An empirical study on LLM-based classification of requirements-related provisions in food-safety regulations. *Empir. Softw. Eng.* **30**(3), 72 (2025)
13. Hoare, C.A.R.: Communicating sequential processes. *Commun. ACM* **21**(8), 666–677 (1978)
14. Jhala, R., Majumdar, R.: Software model checking. *ACM Comput. Surv. (CSUR)* **41**(4), 1–54 (2009)
15. Haonan Li, Yu., Hao, Y.Z., Qian, Z.: Enhancing static analysis for practical bug detection: an LLM-integrated approach. *Proc. ACM Program. Lang.* **8**(OOPSLA1), 474–499 (2024)
16. Li, H., et al.: Retrieval-augmented fine-tuning for improving retrieve-and-edit based assertion generation. *IEEE Trans. Softw. Eng.* (2025)
17. Lin, S.: LLM-driven adaptive source–CSink identification and false positive mitigation for static analysis. In: *Proceedings of the 2025 8th International Conference on Computer Information Science and Artificial Intelligence*, pp. 281–285 (2025)
18. Lin, X., et al.: FVEL: interactive formal verification environment with large language models via theorem proving. *Adv. Neural. Inf. Process. Syst.* **37**, 54932–54946 (2024)
19. Mavin, A., Wilkinson, P., Harwood, A., Novak, M.: Easy approach to requirements syntax (ears). In: *2009 17th IEEE International Requirements Engineering Conference*, pp. 317–322. IEEE (2009)
20. McMillan, K.L.: Symbolic model checking. In: *Symbolic Model Checking*, pp. 25–60. Springer (1993)
21. Nielson, F., Nielson, H.R., Hankin, C.: *Principles of program analysis*. Springer Science & Business Media (2004)
22. Visser, W., Havelund, K., Brat, G., Park, S.J., Lerda, F.: Model checking programs. *Automat. Softw. Eng.* **10**(2), 203–232 (2003)
23. Wang, Y., Guo, S., Wei Tan, C.: From code generation to software testing: AI Copilot with context-based RAG. *IEEE Softw.* (2025)
24. Wiesner, S., Peruzzini, M., Hauge, J.B., Thoben, K.: Requirements engineering. In: *Concurrent Engineering in the 21st Century: Foundations, Developments and Challenges*, pp. 103–132. Springer (2015)
25. Zave, P., Nelson, T.: Validation of formal models: a case study. In: *The Practice of Formal Methods: Essays in Honour of Cliff Jones, Part II*, pp. 292–313. Springer (2024)

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

